



**COMSATS University Islamabad,
Abbottabad Campus**

Project Proposal

(SCOPE DOCUMENT)

for

RomaSub.AI:
Roman Urdu Captions Generator
Version 1.0

By

Muttayyab Abdurrehman	CIIT/FA22-BSE-046/ATD
Muhammad Hashir	CIIT/FA22-BSE-031/ATD
Muneeb Khan	CIIT/FA22-BSE-032/ATD

Supervisor

Dr. Osman Khalid

Bachelor of Science in Software Engineering (2022-2026)

SCOPE DOCUMENT REVISION HISTORY

No.	Comment	Action

Supervisor Signature

Date:

Project Category:

- ☐ **A-** Desktop Application/Information System
- ☐ **B-** Problem Solving and Artificial Intelligence

Table of Contents

ABSTRACT	5
INTRODUCTION.....	5
PROBLEM STATEMENT	5
PROBLEM SOLUTION FOR PROPOSED SYSTEM.....	5
RELATED SYSTEM ANALYSIS	6
ADVANTAGES OF PROPOSED SYSTEM.....	7
SCOPE	7
MODULES	7
MODULE 1: USER AUTHENTICATION.....	7
MODULE 2: VIDEO INPUT HANDLER.....	7
MODULE 3: SPEECH RECOGNIZATION (ASR)	8
MODULE 4: DATASET GENERATION FOR FINE-TUNING	8
MODULE 5: FINE TUNING AND TRAINING	8
MODULE 6: TRANSLITERATION ENGINE.....	8
MODULE 7: REAL-TIME SUBTITLE DISPLAY DURING VIDEO PLAYBACK	8
MODULE 8: SUBTITLE EDITOR.....	8
MODULE 9: EXPORT AND FILE MANAGEMENT.....	8
MODULE 10: FEEDBACK AND LEARNING SYSTEM	9
SYSTEM LIMITATIONS	9
SOFTWARE PROCESS METHODOLOGY	9
TOOLS AND TECHNOLOGIES	9
PROJECT STAKEHOLDERS AND ROLES.....	10
TEAM MEMBERS' INDIVIDUAL TASKS.....	11
DATA GATHERING APPROACH.....	11
CONCEPTS.....	12
GANTT CHART.....	13
MOCKUPS	14
CONCLUSION	17
REFERENCES.....	16
PLAGIARISM REPORT.....	18

Abstract

This project addresses a critical accessibility gap for over 50 million Urdu speakers who use Roman Urdu (Latin script) for digital communication. This innovative approach combines advanced machine learning techniques with practical implementation to deliver a comprehensive captioning solution that serves hearing-impaired users, content creators, and educators effectively.

Introduction

The project proposal document for "Roman Urdu Subtitles" aims to outline the development of a desktop application that automatically generates Roman Urdu subtitles from Urdu speech in videos or audio files. It provides a detailed plan, including system architecture, team details, and anticipated outcomes, to address a significant accessibility gap in Urdu media. The background reveals that with over 50 million Urdu speakers using Roman Urdu for digital communication, there is a lack of tools for direct subtitle generation in this script, impacting hearing-impaired users, educators, and content creators. The system integrates OpenAI [1] Whisper for accurate Urdu speech recognition and a Transformer-based transliteration engine to convert text into Roman Urdu. Designed as a Flutter [2]-based app with a FastAPI backend, it supports real-time and offline subtitle generation, along with editing and export functionalities. The motivation stems from the need to reduce latency and improve accuracy compared to existing multi-step processes. The project is feasible using open-source tools, with an estimated cost of PKR 0–4,000, leveraging available resources like GPUs and test data. By offering a user-friendly solution, it aims to enhance accessibility and contribute to Urdu technology through open-source development.

Problem Statement

The Roman Urdu Subtitles software tackles the problem of inaccessible Urdu media for users who rely on Roman Urdu script, especially hearing-impaired individuals, educators, and content creators facing challenges with manual transcription or inaccurate tools. It resolves this by automating the conversion of Urdu speech from audio or video into timed Roman Urdu subtitles, offering a streamlined and efficient solution. The development is driven by the need to serve over 50 million Urdu speakers who use Roman Urdu for digital communication, filling a gap where no dedicated subtitle tools currently exist. Tools like VEED.IO and Maestra focus on native Urdu or multilingual captions but do not prioritize Roman Urdu, as confirmed by user feedback on social media platforms. Implementing this system provides a valuable learning opportunity to customize AI models for low-resource languages like Urdu, enhancing understanding of speech recognition and transliteration technologies. It also allows hands-on experience in optimizing for desktop environments with offline capabilities. Through this project, the team expects to gain skills in full-stack development using Flutter and FastAPI, as well as proficiency in AI frameworks like PyTorch and Hugging Face [2] for model fine-tuning. Expertise in database management with PostgreSQL [1], and version control with GitHub will be developed. Additionally, building a feedback system for continuous AI improvement will sharpen collaborative and iterative software engineering skills.

Problem Solution for Proposed System

Our system addresses these issues by providing:

- The system solves Urdu media accessibility issues by automating speech-to-subtitle conversion.
- OpenAI [1] Whisper transcribes Urdu audio or video into accurate text.
- A fine-tuned Transformer-based model (M2M100) will be used to transliterate Urdu text into Roman Urdu with improved contextual accuracy.
- Supports both real-time subtitle generation and offline processing.
- Eliminates manual transcription, reducing time and human error.
- A Flutter [2]-based desktop app allows easy subtitle review and editing.
- Multi-format export (SRT, VTT) ensures compatibility with media tools.
- Feedback from users improves transcription and transliteration accuracy.
- Provides a direct, efficient audio-to-Roman Urdu workflow.
- Uses open-source tools for affordability and offline accessibility.
- Enhances inclusion for hearing-impaired users and Roman Urdu readers.
- Bridges the gap between Urdu media and digital accessibility needs.

Related System Analysis

Several existing systems and applications perform automatic speech recognition or subtitle generation, but none are tailored for Roman Urdu users. Tools like YouTube Auto-Captions, Google Translate, and Subtitle Edit provide partial solutions but lack native Urdu or Roman Urdu support. The following table highlights their limitations and explains how the proposed system addresses these gaps.

Table 1: Related System Analysis with proposed project solution

Application Name	Weakness	Proposed Project Solution
YouTube Auto-Captions	Supports only English and a few other major languages; no Urdu or Roman Urdu subtitle support.	The proposed system supports Urdu speech recognition and transliteration to Roman Urdu, enabling localized captions.
Google Translate (Transliteration Feature)	Performs transliteration but lacks real-time synchronization with multimedia and has limited contextual accuracy.	Integrates AI-based transliteration directly with video playback to produce contextually accurate Roman Urdu subtitles in real time.
Subtitle Edit / Aegisub	Requires manual subtitle creation and does not support Urdu speech recognition or Roman Urdu conversion.	Automates subtitle generation from Urdu audio using Whisper ASR and a Transformer-based transliteration model.

Advantages of Proposed System

1. Automatically generates Roman Urdu subtitles from Urdu speech.
2. Enhances accessibility for hearing-impaired users and educators.
3. Provides real-time subtitle display during video playback.
4. Offers an easy-to-use desktop interface for editing and export.
5. Continuously improves accuracy through user feedback and learning.
6. Supports multi-format subtitle export (.srt, .vtt, .txt).
7. Uses open-source, cost-effective tools for sustainable development.

Scope

The scope of the Roman Urdu Subtitles project focuses on creating a user-friendly desktop application designed to generate subtitles in Roman Urdu from Urdu audio or video files. This project includes a simple interface built with Flutter, paired with a FastAPI [6] backend, to ensure smooth operation on desktop computers. It will feature the ability to upload audio or video, convert the spoken Urdu into readable Roman Urdu text, and display subtitles in real-time or after processing. Users can edit these subtitles manually and save them in common formats like SRT or VTT for easy use with other programs. The system will also have a sign-in option to keep user settings safe and a feedback tool to help improve how well it works overtime. Additionally, it will create sample data to make the system smarter by learning from examples. However, this project will not include versions for mobile phones or websites, nor will it use cloud storage or handle multiple speakers talking at once. It is limited to Urdu and Roman Urdu, leaving out other languages. The team will use free tools and basic computer resources, with a small optional cost of PKR 0–4,000 for extra processing power if needed. This keeps the focus on a desktop solution that meets the needs of people who need subtitles, like those with hearing difficulties or teachers, without adding extra complexity.

Modules

The proposed system, RomaSub.AI, is divided into modular components to ensure scalability, flexibility, and ease of maintenance. Each module addresses a specific functionality, while collectively they form a comprehensive subtitle generation and export system. The modular design also allows future enhancements to be integrated smoothly without affecting existing features.

Module 1: User Registration/Authentication

This module handles secure account creation and login. It ensures that only authorized users can access the application's features, protecting data and maintaining system security. It will also serve as part of the business model, allowing only authenticated users to access the full functionality of the system.

Module 2: Video/Audio Input Handler

The video input handler lets users upload or drag-and-drop video/audio files for processing. It supports popular formats like MP4, AVI, and MKV, MP3 and extracts audio automatically for

transcription. A special addition is file size and type validation to ensure smooth operation before sending data to the next step. This makes it easy and reliable for users to start the subtitle process.

Module 3: Automated Speech Recognition (ASR)

This module uses OpenAI [1] Whisper to turn Urdu speech into text. It offers both live and batch transcription options, handling background noise well and creating accurate time-stamped text. A unique feature is its ability to adapt to different speaking styles, providing a solid base for the next stages of subtitle creation.

Module 4: Dataset Generation for Fine-Tuning

This module gathers and organizes Urdu audio, text, and Roman Urdu samples. It cleans and aligns the data to build a useful dataset for training the M2M100 model. A new aspect is its focus on growing a high-quality collection, ensuring the system adapts well to different language styles.

Module 5: Fine Tuning and Training

This module focuses on refining and training the M2M100 model using the dataset generated earlier. It applies user feedback and corrections to improve the transliteration engine's accuracy and naturalness. A special feature is the ability to periodically update the model, ensuring it stays effective as new data is added. This process supports continuous learning, making the system more reliable for diverse Urdu dialects. Users benefit indirectly as the enhanced model improves their subtitle quality over time.

Module 6: Transliteration Engine

The transliteration engine employs a fine-tuned Transformer-based M2M100 model to convert Urdu text into Roman Urdu. It focuses on keeping the meaning and sound of words intact while producing natural text. A special feature is its continuous improvement through training with feedback data, ensuring better accuracy over time.

Module 7: Real-Time Subtitle Display During Video Playback

This module shows Roman Urdu subtitles live as the video plays. It aligns the text perfectly with the audio using timestamps for a smooth experience. A new addition is dynamic updates during playback, making it highly accessible and engaging for users needing captions.

Module 8: Subtitle Editor

The subtitle editor allows users to review and tweak automatically generated subtitles. It includes tools for text fixes, timing adjustments, and a live video preview. A special feature is that it links to the feedback system to save edits for model enhancement.

Module 9: Export and File Management

This module allows users to export subtitles in formats like .srt, .vtt, and .txt. It ensures the exported files work smoothly with popular media players, making it easy to share and use the subtitles anywhere.

Module 10: Feedback and Learning System

The feedback system collects user edits and corrections to improve the system. These changes are used as training data to refine the models, making the AI smarter over time. A special feature is its self-learning capability, adapting to real-world usage for better performance.

System Limitations

1. The system is limited to desktop use and does not support mobile or web platforms, restricting its accessibility to users without suitable devices.
2. It focuses solely on Urdu and Roman Urdu, excluding support for other languages, which limits its versatility.
3. The project relies on available hardware resources and optional Colab Pro access, potentially impacting performance if high-end GPUs are unavailable.
4. Real-time processing may face delays with long videos or poor audio quality, affecting user experience under certain conditions.
5. The system lacks multi-speaker detection, making it less effective for videos with multiple simultaneous speakers.

Software Process Methodology

We will use Object-Oriented Methodology for the development of the Roman Urdu Subtitles project. This approach allows us to design the system as a collection of interacting objects, each representing a module like User Authentication or Video Input Handler, which aligns with our need for modular and reusable code. We chose this methodology because it suits our team's expertise in modern programming languages like Flutter and Python, and it supports the complex interactions between different components, such as speech recognition and transliteration. The choice influences our tools, favoring frameworks like PyTorch and Hugging Face [2], and shapes the System Design Specification (SDS) to emphasize class hierarchies and object relationships. This methodology is ideal for our project as it facilitates scalability and maintenance, especially given the iterative nature of AI model fine-tuning and feedback integration.

Tools and Technologies

The development of RomaSub.AI relies on a combination of modern frameworks, APIs, and development tools to ensure scalability, performance, and usability. Both hardware and software technologies are carefully selected to support cross-platform development, testing, and deployment.

Table 2: Tools and Technologies for Proposed Project

Category	Tool / Technology	Version	Rationale / Purpose
Frontend Development (Desktop)	Flutter	3.10	Cross-platform desktop framework for building the app interface and video playback features.
Backend Development	FastAPI	0.110	High-performance Python web framework for API and AI model integration.
Speech Recognition (ASR)	OpenAI [1] Whisper	2023 Release	Urdu speech-to-text recognition model ensuring high transcription accuracy.
Transliteration Engine	Transformer (M2M100 / IndicXlit)	Latest	Converts Urdu script text into Roman Urdu with context-aware accuracy.
Machine Learning Framework	PyTorch	2.2	Used for fine-tuning AI models and performing inference operations.
NLP Model Library	Hugging Face [2] Transformers	4.39	Provides pre-trained Transformer models for transliteration tasks.
Database Management	PostgreSQL [1]	15.0	Stores feedback, user data, and metadata for model improvement.
Containerization / Deployment	Docker	24.0	Ensures smooth, isolated, and portable deployment across systems.
Version Control	Git & GitHub	Latest	Source code management, collaboration, and version tracking.
Testing / API Verification	Postman	11.54.6	For testing REST APIs and verifying backend model endpoints.
Development Environment	Visual Studio Code	1.104	Code editor for Flutter, FastAPI, and AI development.
Model Training Environment	Google Colab / GPU Setup	Latest	Provides GPU-based cloud environment for training and fine-tuning models.
Documentation & Presentation	MS Office (Word, PowerPoint)	2021	Used for preparing project reports, documentation, and presentations.

Project Stakeholders and Roles

This project involves multiple stakeholders who contribute to its development, evaluation, and eventual usage. Each stakeholder has a specific role, ranging from sponsorship and supervision to implementation and end-user adoption.’

Table 3:Project Stakeholders for Proposed Project

Project Sponsor	COMSATS University Islamabad, Abbottabad campus.
Stakeholder	• Project Supervisor: Dr. Osman Khalid • Developers: Muhammad Hashir, Muttayyab Abdurrehman, Muneeb Khan • Final Year Project Committee • Head of Department / Institute • End Users (Content Creators, Educators, Hearing-Impaired Individuals)

Team Members' Individual Tasks

The project is divided into three members, with each responsible for specific modules based on expertise. This ensures balanced workload distribution and clear accountability for implementation and testing.

Table 4:Team Member Work Division for Proposed Project

Student Name	Student Registration Number	Responsibility / Modules
Muttayyab Abdurrehman	CIIT/FA22-BSE-046/ATD	Responsible for AI and Model Development, focusing on language processing and model fine-tuning. Works on Module 3: Speech Recognition (ASR), Module 4: Dataset Generation for Fine-Tuning, and Module 5: Fine Tuning and Training. Ensures high model accuracy and continuous system improvement.
Muneeb Khan	CIIT/FA22-BSE-032/ATD	Leads Frontend Development using Flutter. Works on Module 1: User Authentication, Module 7: Real-Time Subtitle Display During Video Playback, and Module 8: Subtitle Editor. Designs intuitive UI/UX and ensures seamless real-time subtitle rendering and editing experience.
Muhammad Hashir	CIIT/FA22-BSE-031/ATD	Manages Backend and System Integration, handling server logic, data flow, and deployment. Responsible for Module 2: Video Input Handler, Module 6: Transliteration Engine, Module 9: Export & File Management, and Module 10: Feedback & Learning System.

Data Gathering Approach

We will gather data for RomaSub.AI from multiple authentic and diverse sources to ensure accurate model performance. A major portion of Urdu and Roman Urdu text data will be collected from WhatsApp chats, social media messages, and publicly available chat datasets, capturing real conversational language. Additionally, open-source datasets from Hugging Face [2] and YouTube Urdu videos will be used to collect speech and transcription samples. Since we are native Urdu speakers, we and our friends will also contribute by sharing written and spoken examples of natural Roman Urdu usage. This approach will ensure that the dataset represents genuine linguistic patterns and everyday communication styles.

Concepts

Concept-1: Speech Recognition (ASR)

Automatic Speech Recognition (ASR) involves converting spoken language into text. In RomaSub.AI, the OpenAI [1] Whisper model performs Urdu speech-to-text transcription, forming the base for subtitle generation and transliteration.

Concept-2: Machine Learning and Transliteration

Transliteration converts text from one script to another without changing meaning. RomaSub.AI uses a Transformer-based M2M100 model to transliterate Urdu into Roman Urdu while preserving pronunciation and context.

Concept-3: Deep Learning and Fine-Tuning

This concept focuses on training neural networks using large datasets to improve model performance. Fine-tuning allows RomaSub.AI's transliteration model to learn from user feedback and adapt to various Urdu dialects.

Concept-4: Natural Language Processing (NLP)

NLP enables machines to understand and process human language. In RomaSub.AI, it ensures accurate subtitle generation, context-sensitive translation, and sentence structuring in Roman Urdu.

Gantt chart

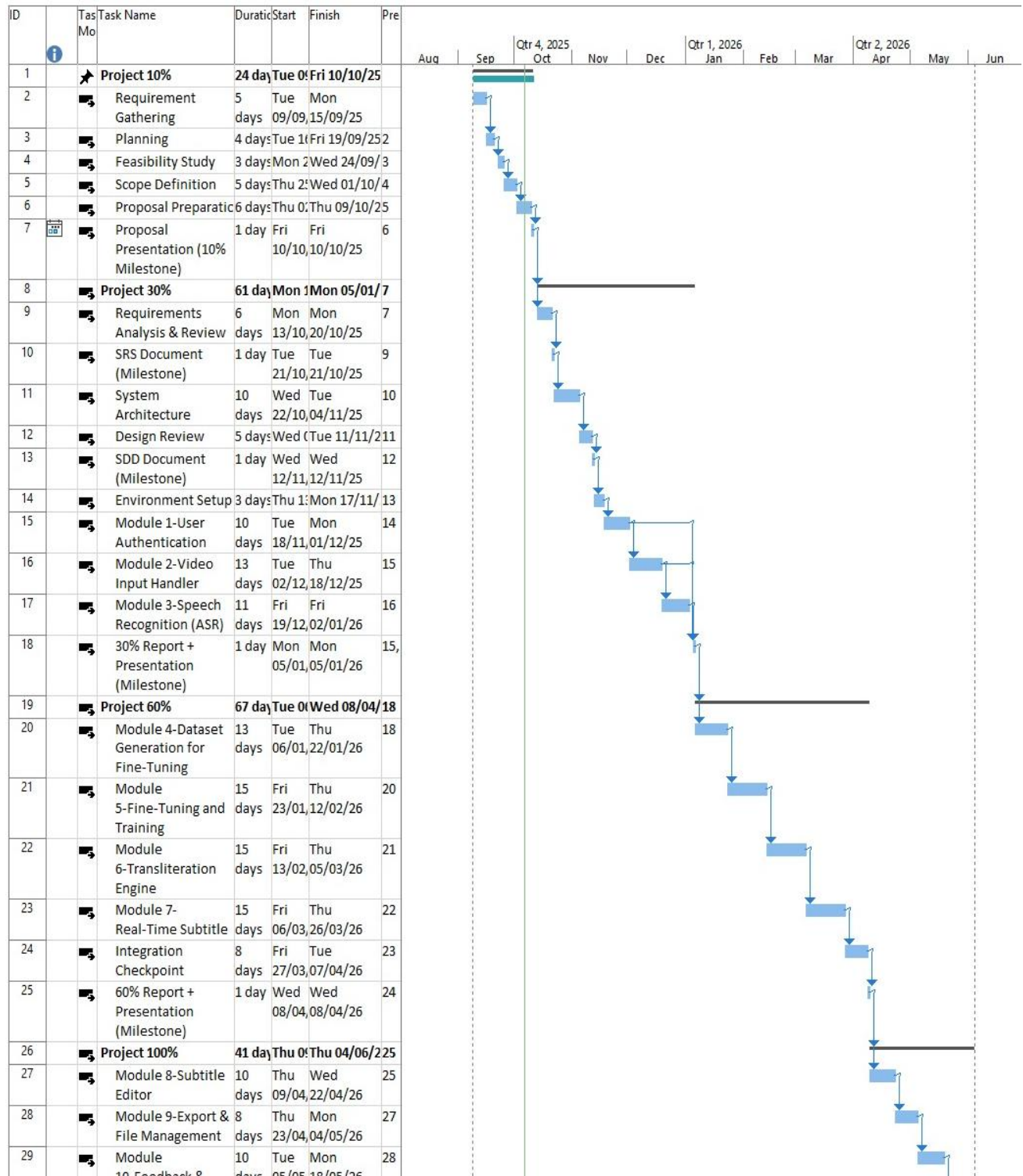


Figure 1:Gantt chart

Mockups

1. Desktop App:

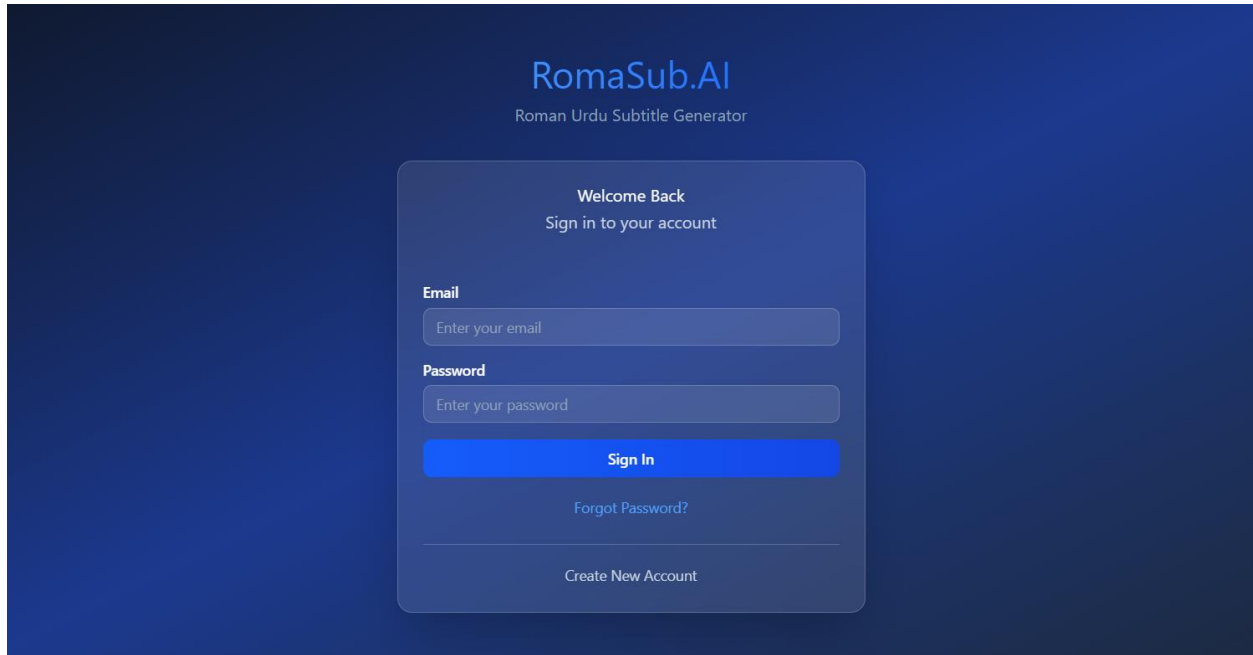


Figure 1: Login Page

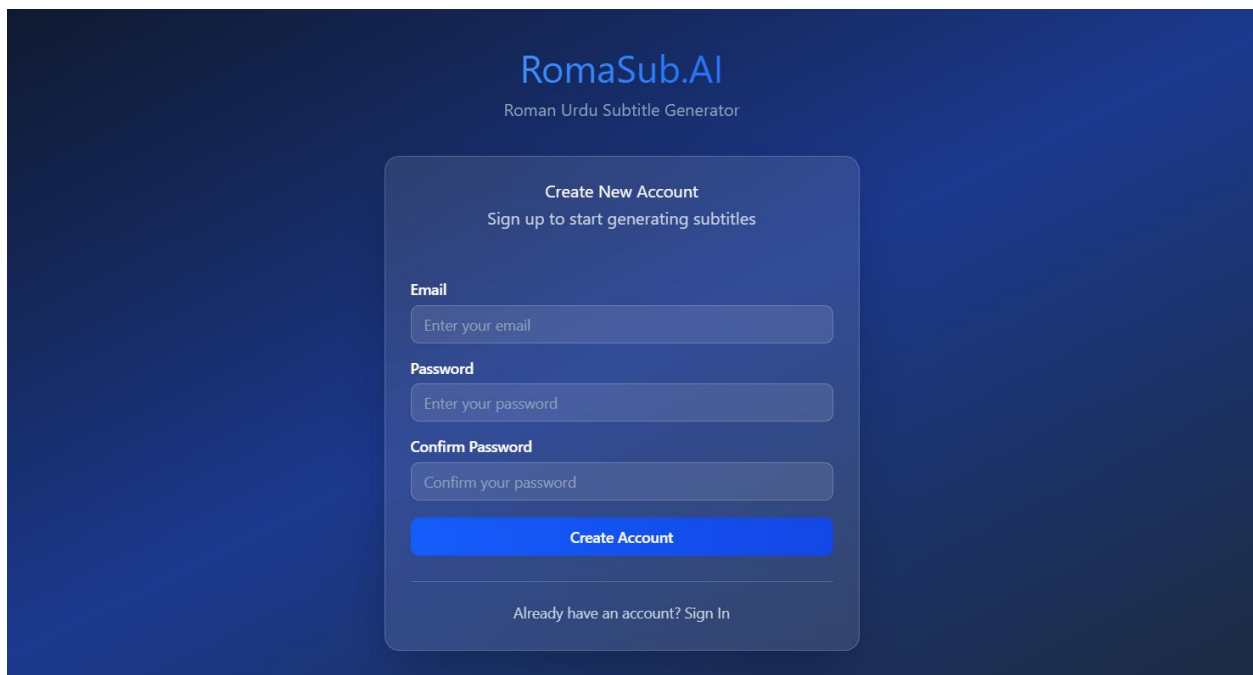


Figure 2: Sign Up Page

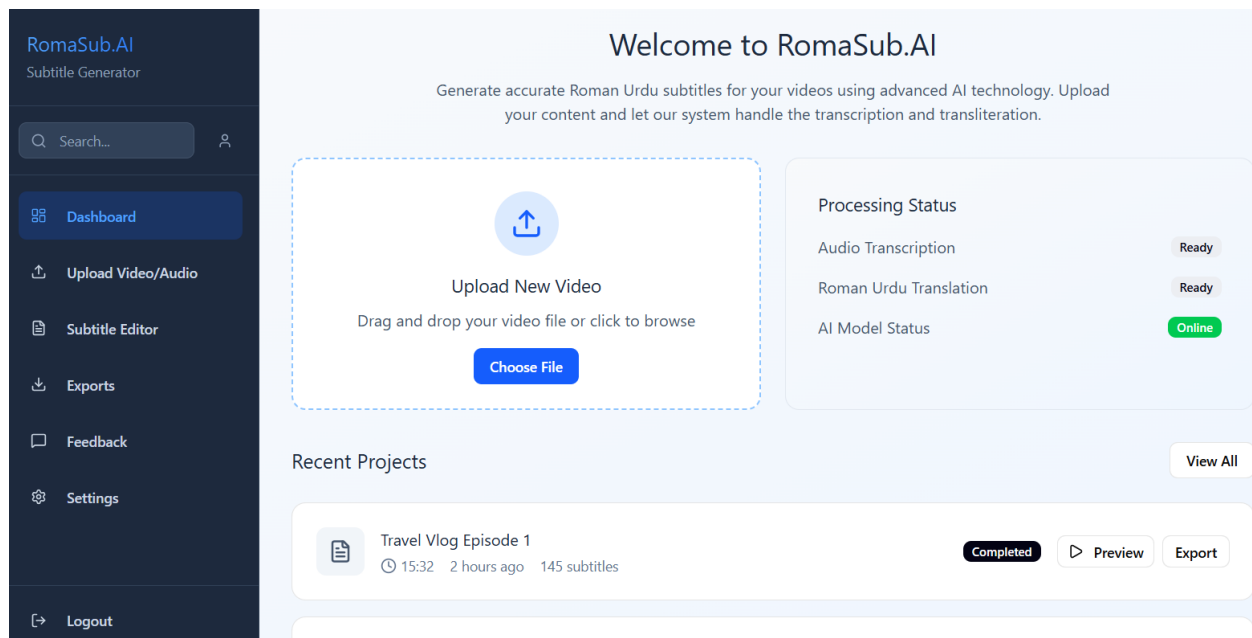


Figure 3: Dashboard

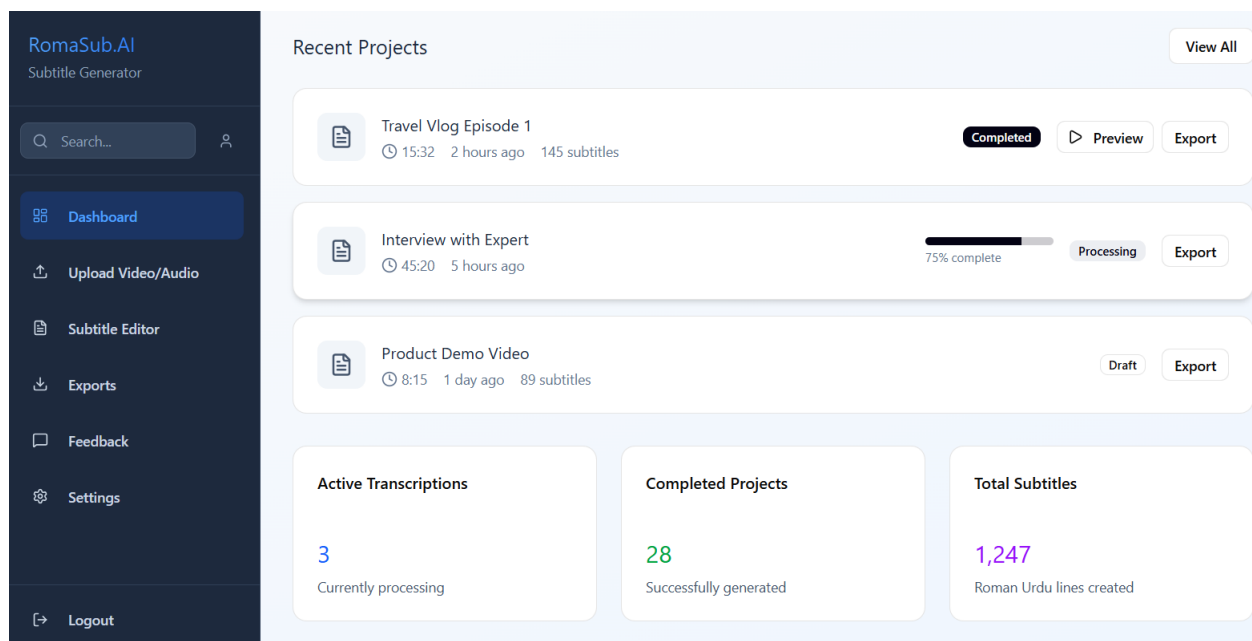


Figure 4: Recent Projects

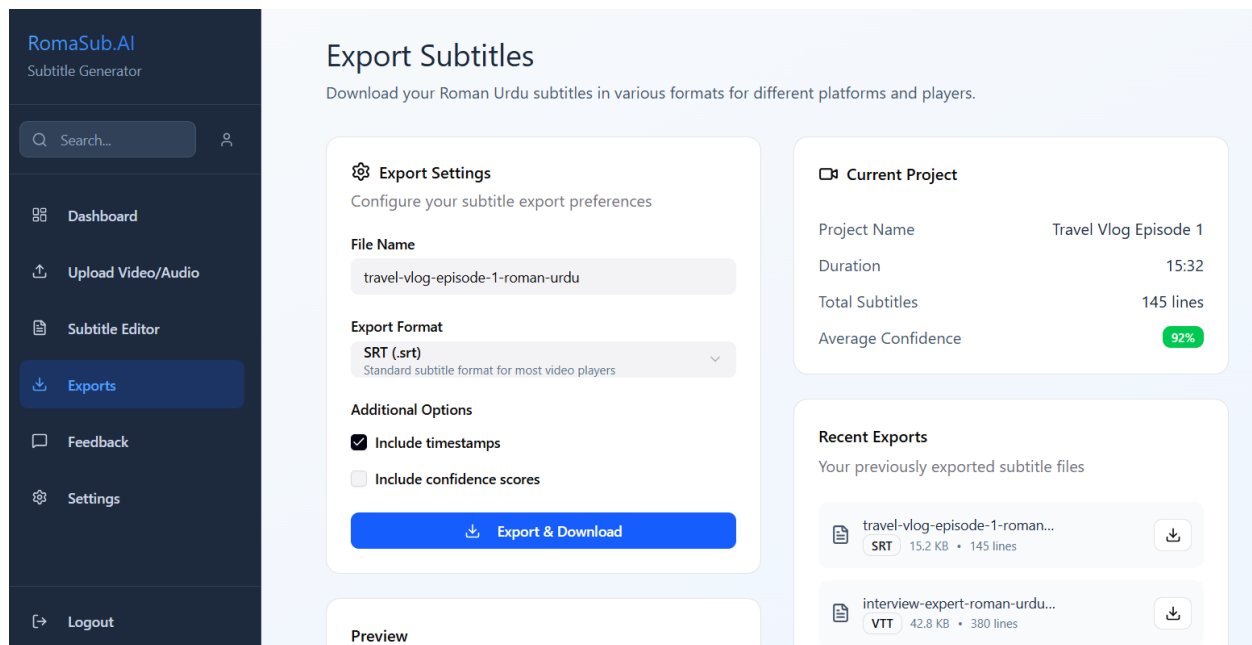


Figure 5: Exports Page

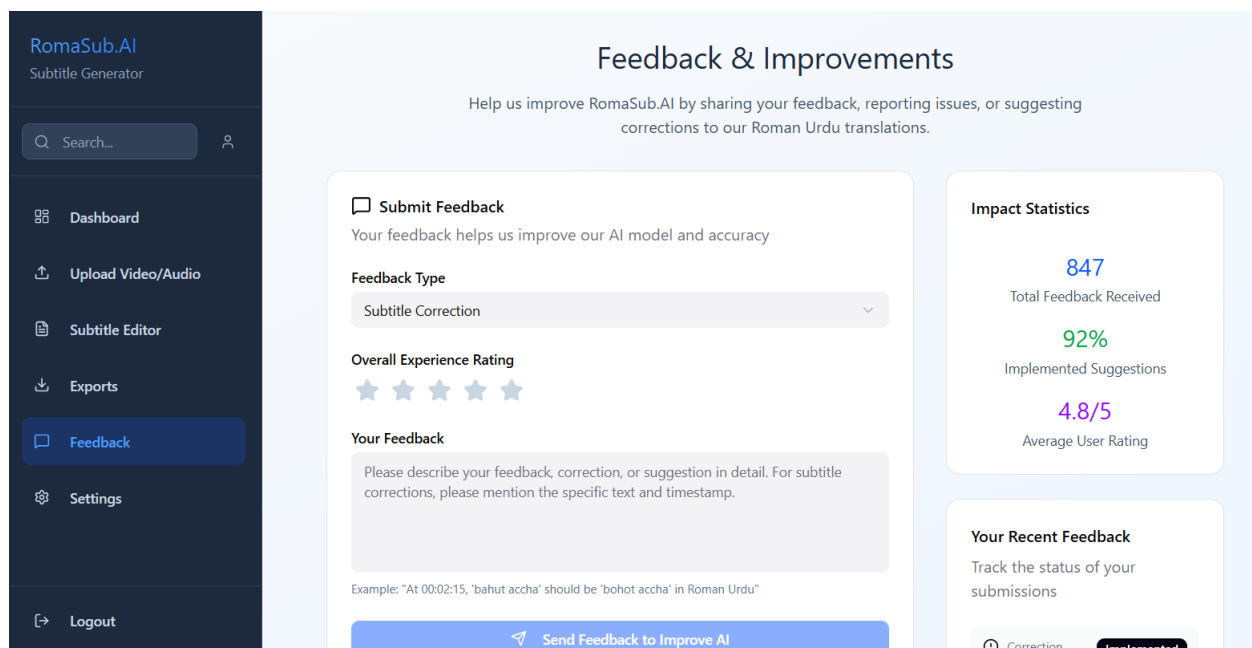


Figure 6: Feedback and Improvements Page

Conclusion

In conclusion, the proposed system RomaSub.AI aims to bridge the accessibility gap for Urdu speakers by providing automated Roman Urdu subtitle generation. By integrating cutting-edge technologies like OpenAI [1] Whisper and Transformer-based transliteration models, the system ensures accuracy, efficiency, and adaptability. The modular design enables real-time processing, easy editing, and continuous improvement through user feedback. This project not only contributes to language technology development but also promotes inclusivity and digital accessibility for millions of Roman Urdu users.

References

- [1] PostgreSQL, “The PostgreSQL Global Development Group,” 25 September 2025. [Online]. Available: <https://www.postgresql.org/docs/>. [Accessed 21 September 2025].
- [2] OpenAI, “OpenAI. Whisper,” 2020. [Online]. Available: <https://github.com/openai/whisper>. [Accessed 28 September 2025].
- [3] “Hugging Face,” [Online]. Available: <https://huggingface.co/docs/transformers>. [Accessed 24 9 2025].
- [4] “Flutter,” 5 December 2019. [Online]. Available: <https://docs.flutter.dev>. [Accessed 15 9 2025].
- [5] “Paszke, A., et al. PyTorch,” 2019. [Online]. Available: Available at: https://papers.nips.cc/paper_files/paper/2019. [Accessed 2 September 2025].
- [6] “FastAPI,” 2019. [Online]. Available: <https://fastapi.tiangolo.com>. [Accessed 12 September 2025].
- [7] “Docker Inc,” [Online]. Available: <https://docs.docker.com>. [Accessed 17 September 2025].

Plagiarism Report

Turnitin Originality Report

Processed on: 07-Oct-2025 5:19 AM PDT

ID: 2773797006

Word Count: 3095

Submitted: 1

Roman Urdu Captions Generator By
Muttayyab Abdurrehman

1% match (student papers from 05-Apr-2019)
[Submitted to Higher Education Commission Pakistan on 2019-04-05](#)

Similarity Index

8%

Similarity by Source

Internet Sources: 7%

Publications: 0%

Student Papers: 5%

1% match (student papers from 26-Mar-2019)
[Submitted to Higher Education Commission Pakistan on 2019-03-26](#)

1% match (Internet from 25-Jul-2024)
<https://WWW.coursehero.com/file/146603028/fypaadocx/>

1% match (student papers from 10-Sep-2018)
[Submitted to Academy of Information Technology on 2018-09-10](#)

< 1% match (student papers from 17-Oct-2018)
[Submitted to Higher Education Commission Pakistan on 2018-10-17](#)

< 1% match (student papers from 28-Sep-2018)
[Submitted to Higher Education Commission Pakistan on 2018-09-28](#)

< 1% match (student papers from 09-Sep-2020)
[Submitted to Higher Education Commission Pakistan on 2020-09-09](#)

< 1% match (Internet from 18-Apr-2025)
<https://www.coursehero.com/file/41077273/Sohail-Malik-FA16-BCS-024FA16-BCS-039-Ver-13docx/>